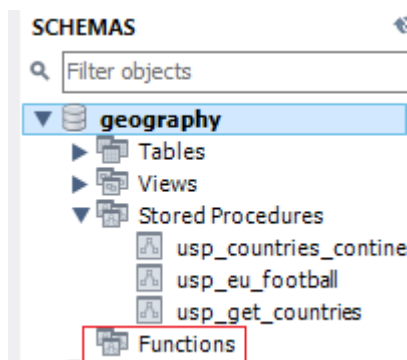


Функции в SQL



Досега сме създавали чрез CREATE таблици, изгледи, процедури.

Какво най-важно запомниме:

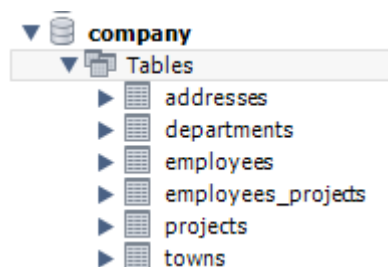
- **CREATE TABLE** – създава таблица. Можем да променяме настройките ѝ с ALTER. Можем да променяме записите ѝ с UPDATE. Можем да изтриваме:

* **DELETE ... WHERE ...** изтрива конкретни записи/редове, които отговарят на някакво условие;

* **TRUNCATE** – изтрива всички записи/редове наведнъж, но запазва настройките. Много по-бърза от DELETE;

* **DROP** – изтрива цялата таблица със записите и настройките в нея.

- **CREATE VIEW** – създаваме изглед с резултат от **стойностите** на дадена заявка. За да се отличават изгледите, по конвенция е желателно да ги именуваме с v_ и описателно име.



Result Grid										
Filter Rows: [] Edit: [] Export/Import: [] Wrap Cell Content: []										
	employee_id	first_name	last_name	middle_name	job_title	department_id	manager_id	hire_date	salary	address_id
▶	1	Guy	Gilbert	R	Production Technician	7	16	1998-07-31 00:00:00.000000	12500.0000	166
	2	Kevin	Brown	F	Marketing Assistant	4	6	1999-02-26 00:00:00.000000	13500.0000	102
	3	Roberto	Tamburello	NULL	Engineering Manager	1	12	1999-12-12 00:00:00.000000	43300.0000	193
	4	Rob	Walters	NULL	Senior Tool Designer	2	3	2000-01-05 00:00:00.000000	29800.0000	155
	5	Thierry	D'Hers	B	Tool Designer	2	263	2000-01-11 00:00:00.000000	25000.0000	40
	6	David	Bradley	M	Marketing Manager	5	109	2000-01-20 00:00:00.000000	37500.0000	199
	7	Jolynn	Dobney	M	Production Supervisor	7	21	2000-01-26 00:00:00.000000	25000.0000	275
	8	Ruth	Ellerbrock	Ann	Production Technician	7	185	2000-02-06 00:00:00.000000	13500.0000	108
	9	Gail	Erickson	A	Design Engineer	1	3	2000-02-06 00:00:00.000000	32700.0000	22
	10	Barry	Johnson	K	Production Technician	7	185	2000-02-07 00:00:00.000000	13500.0000	285
	11	Jossef	Goldberg	H	Design Engineer	1	3	2000-02-24 00:00:00.000000	32700.0000	214
	12	Terri	Duffy	Lee	Vice President of Eng...	1	109	2000-03-03 00:00:00.000000	63500.0000	209

Нека напишем заявка за 3-те имена на служител и неговата заплата:

```
USE company;
```

```
SELECT CONCAT_WS(' ', first_name, middle_name, last_name) AS 'Full Name',  
       salary  
FROM employees;
```

Резултат:

Result Grid			Filter Rows:
	Full Name	salary	
▶	Guy R Gilbert	12500.0000	
	Kevin F Brown	13500.0000	
	Roberto Tamburello	43300.0000	
	Rob Walters	29800.0000	
	Thierry B D'Hers	25000.0000	
	David M Bradley	37500.0000	
	JoLynn M Dobney	25000.0000	
	Ruth Ann Ellerbrock	13500.0000	
	Gail A Erickson	32700.0000	
	Barry K Johnson	13500.0000	
	Jossef H Goldberg	32700.0000	

В момента няма да се впускаме в подреждания и усложняване на заявките.

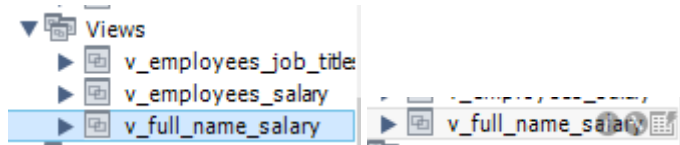
Резултатът от този код може да се запази като изглед с CREATE VIEW име на изгледа, започващо с v_ и AS като псевдоним на кода:

```
1 • USE company;  
2  
3 • SELECT CONCAT_WS(' ', first_name, middle_name, last_name) AS 'Full Name',  
4       salary  
5 FROM employees;  
6  
7 /* view */  
8 • CREATE VIEW v_full_name_salary AS  
9       SELECT CONCAT_WS(' ', first_name, middle_name, last_name) AS 'Full Name',  
10      salary  
11 FROM employees;
```

Макар двата кода да изглеждат еднакви, в първия случай имаме 292 реда засегнати, които веднага се показват като резултат от заявката, а във втория, с изгледа дава зелен верен маркер, но 0 реда са засегнати и видимо нищо не се е случило.

Output				
Action Output				
	#	Time	Action	Message
✓	114	12:10:38	SELECT CONCAT_WS(' ', first_name, middle_name, last_name) AS 'Full Name', salary FR...	293 row(s) returned
✓	115	12:13:22	CREATE VIEW v_full_name_salary AS SELECT CONCAT_WS(' ', first_name, middle_na...	0 row(s) affected

Ако проверим след Refresh, обаче, имаме запазен изглед.



Можем да разгледаме настройките на кода (гаечен ключ) и самия резултат (табличка с гръмотевица):

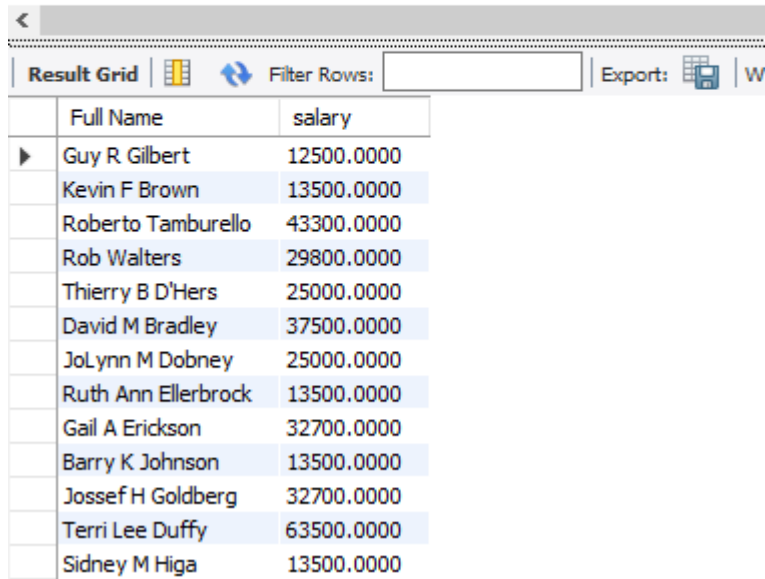
Name: The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```

1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `v_full_name_salary` AS
6     SELECT
7         CONCAT_WS(' ',
8             `employees`.`first_name`,
9             `employees`.`middle_name`,
10            `employees`.`last_name`) AS `Full Name`,
11            `employees`.`salary` AS `salary`
12     FROM
13         `employees`
  
```

```
1 • SELECT * FROM company.v_full_name_salary;
```



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with two columns: 'Full Name' and 'salary'. There are 15 rows of data, each representing an employee. The interface includes a 'Filter Rows' search bar and an 'Export' button.

	Full Name	salary
▶	Guy R Gilbert	12500.0000
	Kevin F Brown	13500.0000
	Roberto Tamburello	43300.0000
	Rob Walters	29800.0000
	Thierry B D'Hers	25000.0000
	David M Bradley	37500.0000
	JoLynn M Dobney	25000.0000
	Ruth Ann Ellerbrock	13500.0000
	Gail A Erickson	32700.0000
	Barry K Johnson	13500.0000
	Jossef H Goldberg	32700.0000
	Terri Lee Duffy	63500.0000
	Sidney M Higa	13500.0000

Т.е. ако искаме да запазим резултат – изглед. Ако искаме да запазим обаче по-сложен код – процедура.

- CREATE PROCEDURE usp_ което е съкратено от User Stored Procedure след което името () – скобите показват възможно подаване на променливи BEGIN код на заявката END. При процедурите е добре да използваме разделител DELIMITER \$\$, защото кодът на по-сложните заявки има няколко ;

```
/* procedure */
```

```
DELIMITER $$
```

```
CREATE PROCEDURE usp_full_name_salary()
```

```
BEGIN
```

```
    SELECT CONCAT_WS(' ', first_name, middle_name, last_name) AS 'Full Name',  
           salary
```

```
FROM employees;
```

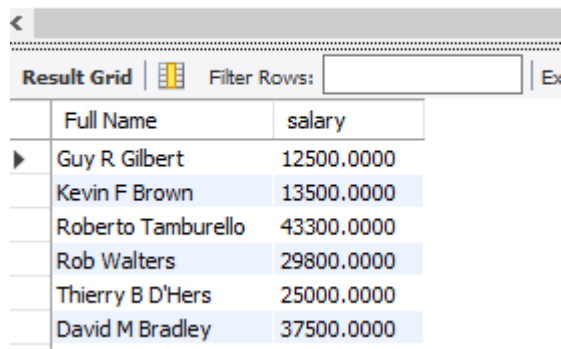
```
END $$
```

Query

Можем да изредим няколко процедури, а когато искаме вече да върнем нормалния разделител, декларираме с DELIMITER ;

Извикването на процедура става с CALL

23 • `CALL usp_full_name_salary();`

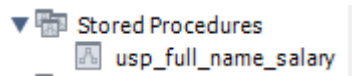


	Full Name	salary
▶	Guy R Gilbert	12500.0000
	Kevin F Brown	13500.0000
	Roberto Tamburello	43300.0000
	Rob Walters	29800.0000
	Thierry B D'Hers	25000.0000
	David M Bradley	37500.0000

Нямаме възможност за Alter – промяна на код, директно Drop + Create.

С какво помагат процедурите:

- * компилират се веднъж, извикват се когато потрѣбват – това спестява натоварване на процесора
- * по-добра логическа структура на самия код
- * спестява се повторение на код
- * по-голяма сигурност
- * не е желателно прекалено трупане на процедури заради паметта.



Отново може да бъде извикана с CALL.

`CALL usp_full_name_salary();`

Последното, което не сме разгледали, са функции – те наподобяват много истинското програмиране с if-else конструкции.

- CREATE FUNCTION udf_ за User Defined Function ()

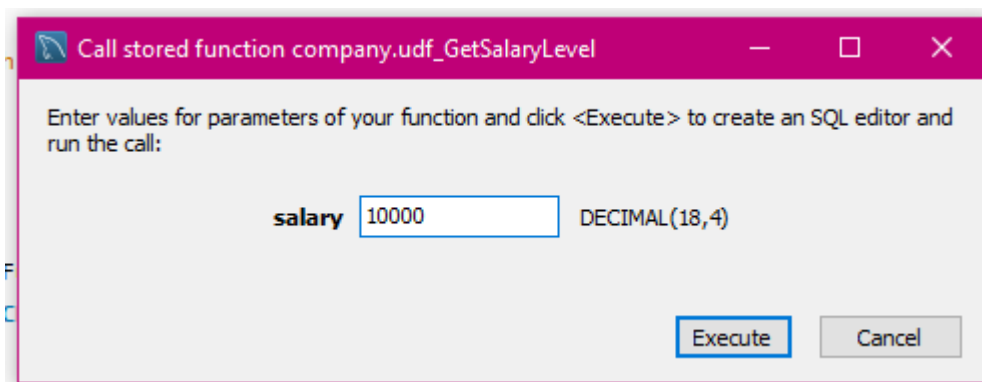
```

/* function */
CREATE FUNCTION udf_GetSalaryLevel(salary DECIMAL(18,4))
RETURNS VARCHAR(10)
DETERMINISTIC не задължително
BEGIN
DECLARE Level VARCHAR(10);
IF (salary < 30000) THEN
SET Level = 'Low';
ELSEIF(salary BETWEEN 30000 AND 50000) THEN
SET Level = 'Average';
ELSE
SET Level = 'High';
END IF;
RETURN Level;
END $$

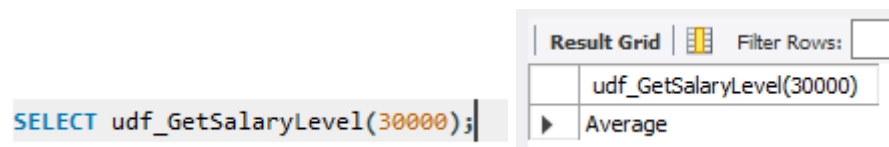
```

По подразбиране е NOT DETERMINISTIC – т.е. при едни и същи входни данни връща различни резултати (изходи). DETERMINISTIC – при един и същи вход връща един и същи изход.

Functions
f() udf_GetSalaryLevel



Т.е. кода, който можем да извикваме:



```
SELECT udf_GetSalaryLevel(70000);
```

Result Grid		Filter Rows:
	udf_GetSalaryLevel(70000)	
▶	High	